Hozi

National Foresight & Decision Infrastructure

First module: Food Security

"Fill the granary before the drought."

Track: Development (Track 3)
Team: Curious Inq
Lead Innovator: Nash Barara
Date: 4 July 2026

Built in Zimbabwe. Open-source. Sovereign.

1. Problem Definition & Strategic Alignment

The Problem

Almost every year, parts of Zimbabwe slide into food shortage when the rains fail. The tragedy is rarely that no one saw it coming — it is that the warning arrives too late, buried in slow seasonal reports produced outside the country, after the moment for early action has passed. Planners are then left to stretch limited resources across many struggling districts with no quick, shared way to see where action would do the most good.

Prediction is a crowded field. Prescription is not. Global tools (WFP HungerMap, FEWS NET) tell you *what* will happen. None of them help our own planners decide *where to act first* — interactively, in local languages, and sovereign. Hozi closes that gap: **earlier warning + a clear, shared picture + help deciding where each dollar protects the most people.**

Target Users and Beneficiaries

Primary Users	Beneficiaries
• ZimVAC field members and secretariat staff • Ministry of Agriculture / Agritex district and provincial planners • District development officers • Department of Civil Protection duty officers	Rural populations in Zimbabwe's drought-prone districts — particularly the most food-insecure households in Matabeleland, Masvingo, Mashonaland East, and Manicaland provinces — who benefit from earlier, better-targeted anticipatory action.

One Engine, Many Decisions

The pattern Hozi runs — *signals → risk score → forecast → response ranking → local-language playbooks* — is domain-agnostic. Food security is the proven first module: real-outcome training done, baselines beaten, trilingual playbooks shipped. The next two modules are already on the roadmap:

- **Health:** cholera/typhoid outbreak risk by district (rainfall + water points + case reports).
- **Climate/disaster:** flood displacement risk (Cyclone Idai lesson; CHIRPS + river gauges + settlements).

Same engine, new signal set, new institutional customer. That is the infrastructure play.

Strategic Alignment

National AI Strategy 2026–2030

Hozi is designed as the **decision layer** on top of **Project Pangolin** (Zimbabwe's national data platform, a National AI Strategy initiative). Pangolin owns the data; Hozi turns it into decisions. The system is built entirely on open-source tools (Python stdlib, n8n, MIT licence), aligns with the Strategy's goals of sovereign AI capacity, local language technology, and Zimbabwean ownership of analytical infrastructure.

AI4I Priority Sectors

AI4I Priority Sector	Hozi Alignment
Agriculture / Food Systems	Core wedge module — district-level food-security foresight and response planning

Climate Resilience	Roadmap module — flood displacement foresight (same engine, climate signals)
Health	Roadmap module — cholera/typhoid outbreak risk (same engine, health signals)
Local Language Technologies	Trilingual interface and playbooks: English, chiShona, isiNdebele (60 district briefs shipped)
Public Service Delivery	Response Planner ranks where limited support protects the most people — decision support for public resource allocation

National Development Strategy

Hozi directly supports NDS priorities in food security, climate adaptation, and digital transformation. The open-source, sovereign design ensures no dependence on foreign agency access or proprietary data pipelines.

Team: Curious Inq (2 members)

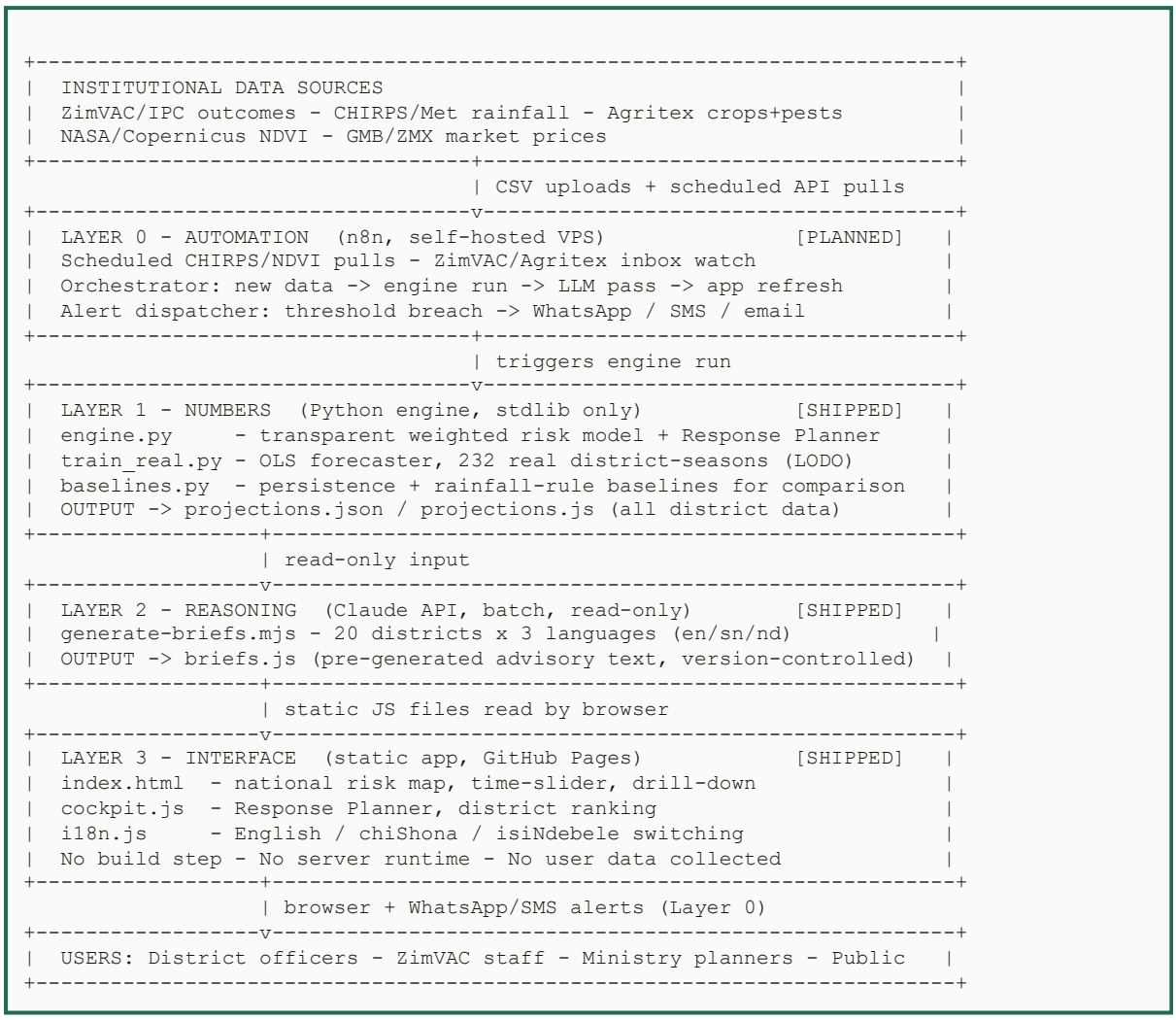
Nash Barara — Product & design lead. 11+ years in advertising, 3D design, and motion graphics. AI-assisted build (fully disclosed).

Chipo — Strategy & copywriting.

Built with AI assistance: Claude served as pair developer throughout — writing engine code, tests, and documentation from specifications authored by the team. Fully disclosed in `docs/AI-USAGE-NOTE.md`. The AI4I Development Track ToR explicitly permits AI-assisted coding provided the team can explain and defend the system.

2. Technical Design & Product Logic

System Architecture: Four Layers



Component Table

Component	Technology	Responsibility	Status
Layer 0: n8n automation	n8n (open-source, Docker)	Scheduled ingestion; orchestrates engine + LLM runs; WhatsApp/SMS/email alerts	Planned
engine.py	Python 3.12, stdlib only	Transparent weighted risk model; Response Planner ranking	Shipped
train_real.py	Python 3.12, stdlib only	OLS forecaster on 232 real IPC × CHIRPS observations; LODO validation	Shipped
baselines.py	Python 3.12, stdlib only	Persistence + rainfall-rule baselines for comparison	Shipped
generate-briefs.mjs	Node.js ESM; Claude API (claude-sonnet-4-6)	Batch playbook generation: 20 districts × 3 languages	Shipped
Static app	Vanilla HTML/CSS/JS, Leaflet.js	Risk map, time-slider, drill-down, Response Planner, honesty panel	Shipped

Two-Layer AI Design

Layer 1: Deterministic Numbers (Classical ML)

The Python engine computes every number in the app through traceable arithmetic. The OLS forecaster is trained on a real panel of 232 district-season observations (IPC Phase 3+ outcomes × CHIRPS dekadal rainfall), validated leave-one-district-out (LODO). **The LLM never generates, edits, or imputes any data point.**

Baseline comparison (LODO, real IPC × CHIRPS panel, 232 obs, 58 districts):

Method	r	MAE (pp)
Global-mean	−0.557	10.2
Persistence (prior-season own district)	0.129	11.0
Rainfall-rule (threshold heuristic)	−0.179	10.7
OLS model (Hozi)	**0.484**	**8.5**

Every simpler method loses on both metrics. r = 0.484 is moderate — honest, not high-confidence. This is foresight, not fortune-telling. The model uses rainfall as its sole feature; adding NDVI, market prices, and a longer label time-series (15+ years of ZimVAC assessments are digitisable) is the documented next step.

Layer 2: Read-Only LLM (Claude API, claude-sonnet-4-6)

Claude operates **only after** Layer 1 has finished. It receives completed district data as read-only input and performs three functions:

1. **REASONS** — plain-language driver explanations for each district's risk score.
2. **SUGGESTS** — ranked intervention playbooks in en/sn/nd (60 playbooks shipped), grounded only in supplied inputs, labelled advisory.
3. **VALIDATES** (design intent) — contradiction-flagging between model output and IPC classifications. Documented architecture; not yet in the current generation script.

Guardrails: Advisory-only output. Never feeds back into Layer 1 scores. Outputs committed to repo as auditable `briefs.js`. Zero API calls at browse time. Human review required before action. ZimVAC/ministry retains decision authority.

Data Schemas, API Layer & Connectivity

Input panel CSV: `district, period, r1h, r3h, rfq, r3q, food_insecurity_pct` (IPC Phase 3+ %). 232 obs, 58 districts, 4 periods (2019–2020). **Output:** `projections.json` — per-district object with risk series (0–100, observed/forecast flag), drivers, forecast slope+band, support-package effect, Response Planner rank. Documented, machine-readable JSON consumed by app and `generate-briefs.mjs`. HTTP REST wrapper is a pilot milestone. Integration with Project Pangolin requires only feed credentials and n8n connector configuration.

Low-connectivity: n8n dispatches WhatsApp/SMS alerts on threshold breaches (2G-capable, no app install). Static app is pre-computed; zero runtime API calls from browser.

> **Edge deployment: N/A.** Server-side processing + static web delivery. No on-device inference.

3. Deliverables & CCE Implementation Roadmap

Development Timeline

Phase	Days	Deliverables
Repo hardening & baselines	1–2	Unit tests (14 stdlib tests); dependency manifests; secrets audit; baseline experiment (LODO comparison)
Proposal & dataset statement	3–5	10-page proposal PDF; dataset provenance statement; AI usage note
Business & deployment pack	6–7	Business model one-pager; deployment plan; demo hosting; n8n workflow
Pitch & evidence	8–9	Pitch deck; testing evidence capture; ToR self-assessment checklist
Review & submit	10–11	Chipo review pass; PDF formatting; portal registration; submission with buffer

Post-Submission Milestones

Phase	Milestone	Deliverable
Days 0–30 (Post-submission)	Foundation	• Incorporate judge and shortlist feedback • Complete security and compliance checklist (DPA review, .env audit) • Sign pilot MOU with at least one ZimVAC/ministry stakeholder • Deploy public demo URL on GitHub Pages
Days 31–60 (Live pilot)	Pilot launch	• Live pilot in Matabeleland South (Beitbridge, Gwanda, Mangwe) • n8n automation running: CHIRPS pull → engine run → WhatsApp alerts • Collect alert delivery receipts and officer feedback • Web-based role gating for admin views
Days 61–90 (Evidence & scale)	Adoption case	• Measure warning lead-time vs ZimVAC seasonal assessment cycle • Complete adoption case study (districts covered, alerts delivered, officer feedback) • Draft provincial scale proposal • Publish refined model with expanded ZimVAC/IPC training labels • Government handover runbook and operator documentation

Compute Requirements

Requirement	Specification
GPU	None required. All computation is CPU-only (OLS linear algebra, weighted arithmetic).
Python runtime	Python 3.9+ (stdlib only). Containerisable as `python:3.12-slim`.
Node.js	Node.js ≥18 (for `generate-briefs.mjs` only). Zero npm dependencies.
Automation	`n8nio/n8n` official Docker image. Self-hosted on 2 GB VPS.

External API	Claude API (claude-sonnet-4-6) for batch playbook generation. ~US$0.10–0.40 per full run.

Dependency-Locked Manifests

Manifest	Content
`requirements.txt`	Zero third-party packages. Engine uses Python stdlib only (`csv`, `json`, `math`, `os`, `statistics`, `sys`). Eliminates supply-chain risk entirely.
`scripts/package.json`	`"dependencies": {}`. Node.js built-ins only (`fs`, `https`). Zero `node_modules`.

ZCHPC CCE Deploy & Test Plan

The system is containerised and ready for handover to ZCHPC CCE if a hosting arrangement is agreed. No hosting agreement is in place; this is the documented scale pathway.

Smoke test on any environment (including CCE):

1. Engine run: `python engine/engine.py` — produces `projections.json` for all districts.
2. Unit tests: `python -m unittest discover -s tests` — 14 tests, all pass in <1 second.
3. One n8n workflow execution: scheduled CHIRPS pull → engine trigger → alert message.

Container images: `python:3.12-slim` (engine) + `n8nio/n8n` (automation). No GPU. No proprietary runtime.

4. Compliance & Risk Mitigation

Data Protection Act [Chapter 12:07] Compliance

Current prototype: No personal data is processed at any stage. All inputs are district-level aggregate statistics — published IPC Phase 3+ percentages, CHIRPS dekadal rainfall averages, ZimVAC seasonal assessment figures, and POTRAZ synthetic challenge data. The unit of analysis is the administrative district, not the individual. Exposure under Chapter 12:07 is minimal by design.

Pilot: Consent Checkbox Implementation (Designed, Not Yet Shipped)

The pilot roadmap includes a district-officer ground-truth reporting channel. Because officer submissions could constitute personal data, the following DPA safeguards are designed:

DPA Requirement	Implementation
Informed consent	Required, **unticked-by-default** checkbox on the report form. Text: "I consent to Hozi storing this submission — district, indicator, date, and my officer ID — for food-security monitoring. I can request deletion by contacting [contact address]." Form cannot submit without checking.
Timestamped consent record	Each submission stores: officer ID (not full name), district, indicator, value, timestamp, and `consent_given: true / timestamp`. Consent record stored alongside submission.
Data minimisation	Three fields only: district, indicator type+value, date. No full name, home address, or phone number beyond minimum officer ID.
Right to erasure	Named contact address in consent notice. Deletion within five working days, confirmed by reply.
Retention limit	Pilot duration + 12 months, then purged unless incorporated into anonymised aggregate data.

Ethical Safeguards & Responsible AI

Risk	Mitigation
Over-trust in forecasts Planners treat risk scores as precise measurements	Confidence band widens with forecast horizon. Forecast months shown with dashed lines and band shading. Mandatory closing note on urgency and limits in every playbook. In-app honesty panel states "Foresight, not fortune-telling." Known limitations listed in README.
LLM hallucination Claude invents figures or programmes not in district data	Read-only two-layer architecture: LLM output never feeds back into Layer 1. System prompt: "Ground EVERY statement in the numbers provided. Never invent figures, places, or programmes." Outputs version-controlled in `briefs.js`. In-app label: "Drafted by AI from the engine's risk data — for planner review, not instruction."
Bias / underserved districts Districts with sparse data silently scored	LODO cross-validation across all 58 districts. Districts with fewer than 3 observed months receive no forecast and are excluded from Response Planner ranking — not silently scored.
Misuse as budget authority Planner output misread as allocation instruction	Explicit framing throughout: "Hozi never sets budgets — it helps stretch the ones that exist." Response Planner is a prioritisation aid, not an instruction. ZimVAC/ministry retains full decision authority.

Cybersecurity Protections

Control	Status
No secrets in repository	Implemented. Git history audited 2026-07-03. API key loaded from gitignored file; `.env.example` committed.
Zero-dependency supply chain	Implemented. Python: stdlib only. Node.js: `"dependencies": {}`. No `node_modules`. Eliminates supply-chain attack surface.
Static frontend, minimal attack surface	Implemented. No server-side code, no user input stored, no cookies, no runtime API calls at browse time.
Pilot VPS hardening	Planned. SSH-key-only access, UFW firewall (ports 22/80/443), unattended security upgrades, n8n behind reverse-proxy auth.
HTTPS	Implemented (GitHub Pages). Planned (pilot VPS via Let's Encrypt).

Model Unit Testing

14 unit tests across two test files, all Python stdlib `unittest`, zero external dependencies:

- **test_engine.py** (12 tests): clamp bounds, risk model (healthy <10, drought >80, irrigation mitigates monotonically, output bounded 0–100), Pearson r (perfect positive/negative/constant), OLS linfit + band thresholds.
- **test_train_real.py** (2 tests): OLS solver recovers known coefficients to 6 decimal places; `predict()` matches manual dot-product.

Run: `python -m unittest discover -s tests`. Expected: 14 pass in <1 second. Validation beyond unit tests: LODO baseline experiment (rerunnable via `python engine/baselines.py`).

5. Sustainability & Future Adoption

Cost Projections

Item	Est. Monthly (US$)
VPS — n8n automation + Python engine (2 GB cloud server)	20–40
Static app hosting (GitHub Pages)	0
Claude API — reasoning + playbook generation (~20 districts/month)	30–80
WhatsApp/SMS alert delivery (pilot volume)	10–30
Domain name + automated backups	5
Total (pilot)	**65–155**

3-month pilot: approximately US$200–500 infrastructure + field onboarding costs. **National scale:** ZCHPC CCE hosting replaces the VPS, reducing cash cost — subject to hosting arrangement being agreed.

Operational Business Model

Phase	Mechanism
Pilot (0–12 months)	Development partner programme grant (WFP/FAO/UNDP). Government operates; partner funds.
Scale (12+ months)	Per-domain institutional licence. Health module → Ministry of Health. Climate module → Dept. of Civil Protection. Each new domain = new institutional customer on the same infrastructure.
Long-term	Government line-item adoption + ZCHPC national hosting. Operating cost moves off grants onto National AI Strategy budget lines.

The open-source MIT engine lowers adoption barriers and supports trust-building with conservative institutions. Curious Inq provides design, integration, and deployment expertise as the incubation operator; the explicit goal is government handover at national scale.

Licensing Registry

Asset	Licence
Hozi engine + app code	MIT (free to use, extend, build upon)
CHIRPS rainfall data	Creative Commons (open)
IPC / ZimVAC outcome labels	Public government and UN agency reports
POTRAZ programme dataset	Programme terms (challenge-provided; not for redistribution)
n8n automation	Sustainable Use License (open-source, self-hostable)
Claude API	Anthropic commercial API terms

Scaling Pathways

Stage	Scope	Trigger
Pilot	1 district cluster (~3 districts, 10–20 users)	MOU signed; infrastructure deployed
Provincial	10–20 districts, one province	Pilot adoption case study complete

National	All ~60 districts, multiple ministries	ZCHPC CCE handover; government operating budget
Multi-domain	Health, climate/disaster, water modules	Each new institutional customer signs module agreement

Live demo: https://nashbarara.github.io/hozi (activation in progress)
GitHub: https://github.com/nashbarara/hozi

"Fill the granary before the drought."